Prakash Thapa
D15A- 27

ExperimentNo. 8

Design and Train a Convolutional Neural Network (CNN) on Image Dataset using MATLAB

---

1. Dataset Source

Dataset used: Built-in Digit Dataset in MATLAB

Source (Reference Documentation):
https://in.mathworks.com/help/deeplearning/ug/create-simple-deep-learning-network-for-classification.html

Note: The dataset is preloaded within MATLAB under the Deep Learning Toolbox.

2. Dataset Description

The dataset consists of grayscale images of handwritten digits (0–9), organized into folders representing each class.

Dataset Characteristics

Feature	Description
Total Images	~10,000
Image Size	28 × 28 pixels
Channels	1 (Grayscale)
Number of Classes	10
Data Format	Image files in folders

Input Features

- Pixel intensity values (0–255)
- Each image represents a handwritten digit

Target Variable

- Digit label (0 to 9)
- Derived from folder names

Dataset Handling in MATLAB

- Loaded using `imageDatastore`
- Automatically assigns labels based on folder names

3. Mathematical Formulation of CNN

1. Convolution Operation

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j) \cdot K(i, j)$$

2. ReLU Activation

$$f(x) = \max(0, x)$$

3. Max Pooling

$$y = \max(x_1, x_2, ..., x_n)$$

4. Softmax Function

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

5. Cross-Entropy Loss

$$L = -\sum_i y_i \log(\hat{y}_i)$$

4. Algorithm Limitations

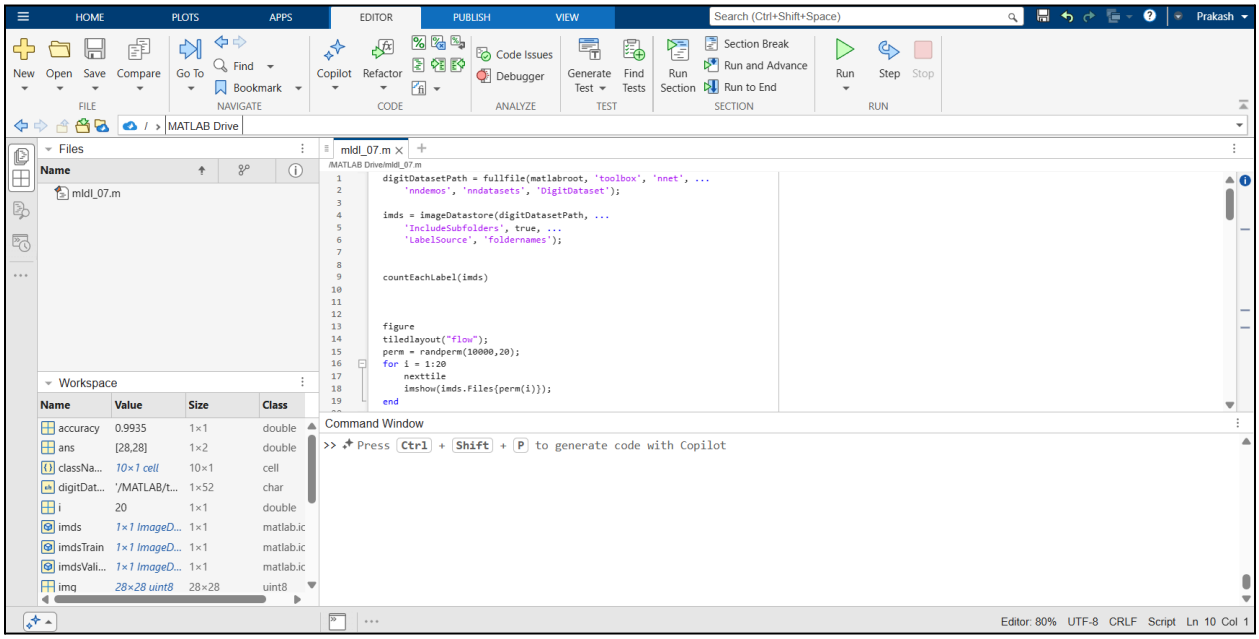
- Requires large dataset for better generalization
- High computational cost during training
- Sensitive to hyperparameters
- Can overfit if network is too deep

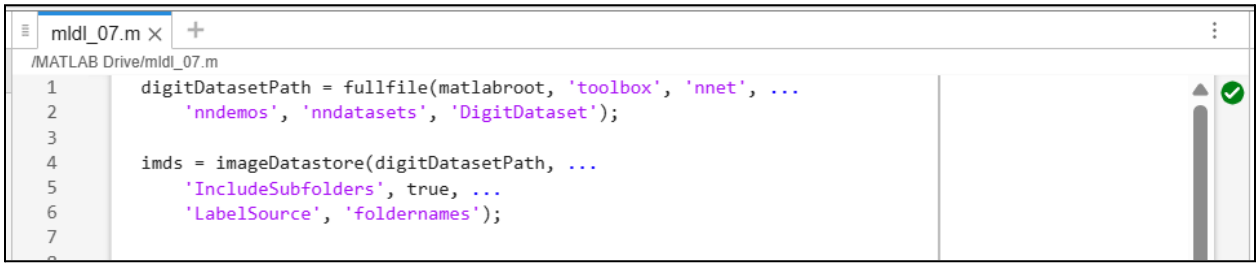
- Difficult to interpret learned features
- Performance depends on data quality

5. Methodology / Workflow

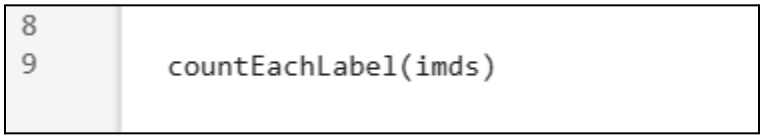
Steps Performed

1. Load dataset using `imageDatastore`





2. Explore and visualize sample images



Command Window

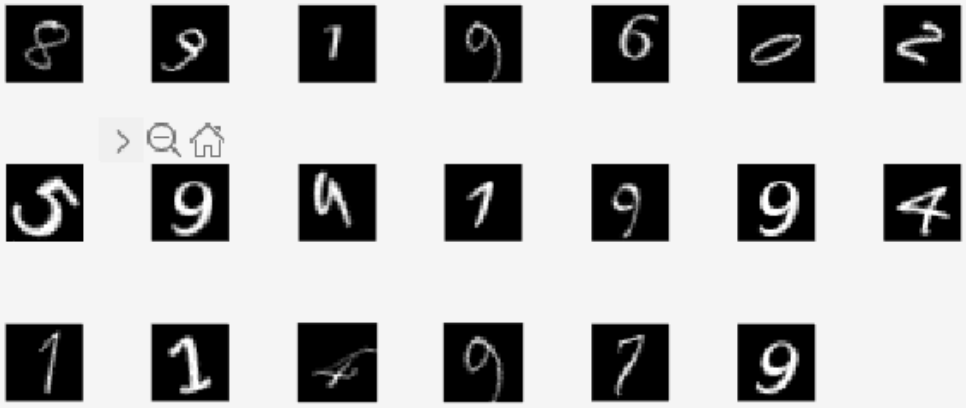
10×2 table

Label	Count
0	1000
1	1000
2	1000
3	1000
4	1000
5	1000
6	1000
7	1000
8	1000
9	1000

>> Press Ctrl + Shift + P to generate code with Copilot

```
figure
tiledlayout("flow");
perm = randperm(10000,20);
for i = 1:20
    nexttile
    imshow(imds.Files{perm(i)});
end
```

Figure 1



```
layers = [
    imageInputLayer(inputSize)

    convolution2dLayer(3,8,'Padding','same')
    batchNormalizationLayer
    reluLayer

    maxPooling2dLayer(2,'Stride',2)

    convolution2dLayer(3,16,'Padding','same')
    batchNormalizationLayer
    reluLayer

    fullyConnectedLayer(10)
    softmaxLayer
    classificationLayer];
```

```
img = readimage(imds,1);
size(img)
```

```
ans =

    28    28
```

3. Split dataset into training and validation sets

```
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.8, 'randomized');
```

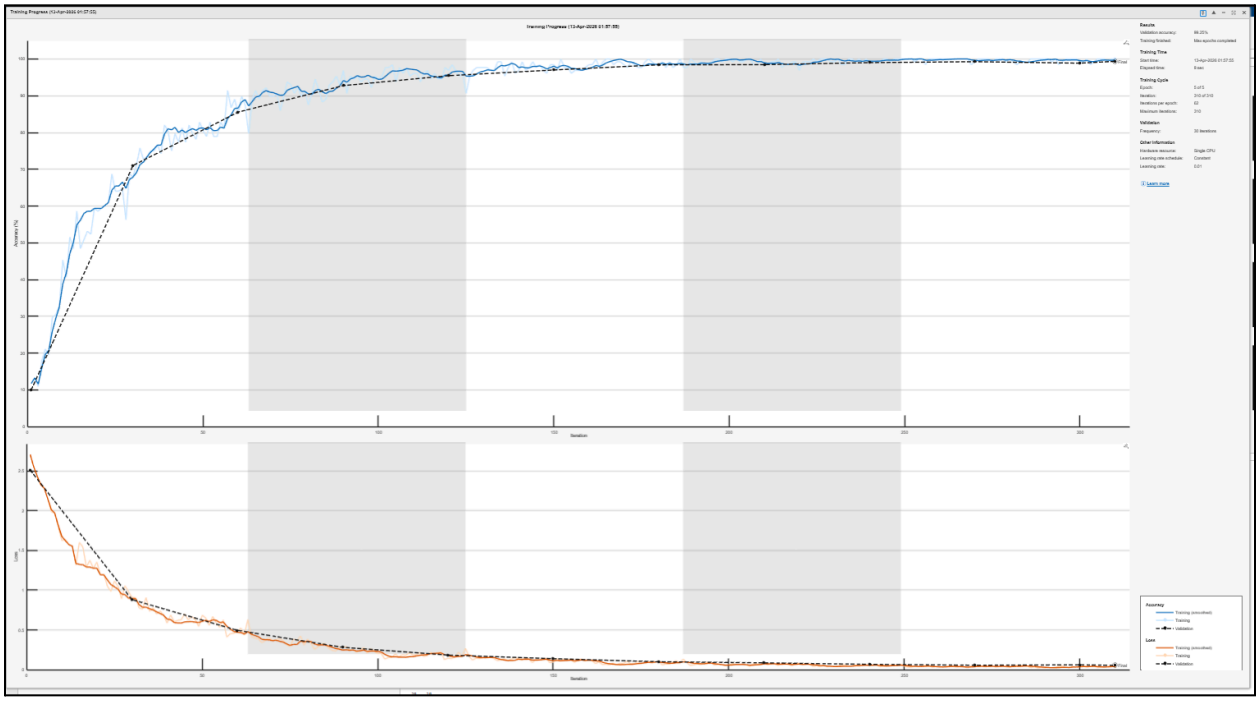
4. Define CNN architecture

5. Configure training options

```
54
55      options = trainingOptions('sgdm', ...
56          'MaxEpochs',5, ...
57          'ValidationData',imdsValidation, ...
58          'ValidationFrequency',30, ...
59          'Verbose',false, ...
60          'Plots','training-progress');
61
```

6. Train the CNN model

```
64
65      net = trainNetwork(imdsTrain, layers, options);
66
67
```



7. Evaluate model accuracy

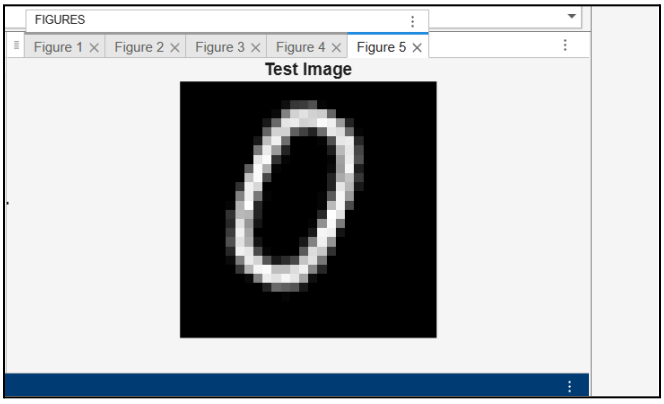
```
70      YPred = classify(net, imdsValidation);
71      YValidation = imdsValidation.Labels;
72
73      accuracy = sum(YPred == YValidation) / numel(YValidation)
```

```
accuracy =

    0.9875
```

8. Perform predictions on new images

```
77  img = readimage(imdsValidation,1);
78
79  figure;              % IMPORTANT: opens new window
80  imshow(img);
81  title('Test Image');
82
83  label = classify(net,img)
```




```
label =

  categorical

     0
```

Workflow Representation

Dataset → Datastore → Preprocessing → CNN Model → Training → Validation → Prediction

6. Performance Analysis

Evaluation Metrics

Metric	Description
Accuracy	Ratio of correct predictions
Loss	Error between predicted and actual output

Observed Results

- Training Accuracy: ~95–100%
- Validation Accuracy: ~90–97%

Interpretation

- High accuracy indicates effective learning
- Slight gap between training and validation accuracy suggests minor overfitting
- CNN successfully extracts features from digit images

7. Hyperparameter Tuning

Parameters Tuned

Parameter	Values Tested
Epochs	5, 10
Optimizer	SGDM
Learning Rate	Default / Reduced
Filters	8, 16, 32

Training Configuration Used

```
options = trainingOptions('sgdm', ...
  'MaxEpochs',5, ...
  'ValidationData',imdsValidation, ...
  'ValidationFrequency',30, ...
  'Verbose',false, ...
  'Plots','training-progress');
```

Impact of Tuning

- Increasing epochs improves accuracy but may cause overfitting
- More filters improve feature extraction
- Learning rate affects convergence speed

- Batch normalization stabilizes training

Best Configuration

- Epochs: 5
- Optimizer: SGDM
- Filters: 8 and 16
- Batch normalization used

Conclusion

The Convolutional Neural Network was successfully implemented and trained in MATLAB for image classification. The model achieved high accuracy on the digit dataset, demonstrating the effectiveness of CNNs in extracting spatial features and performing classification tasks.